Task 1: Guided Activity, A custom delegate:

```
import Foundation
protocol ColorPickerDelegate: AnyObject {
func didChooseColor(_ color: String)
}
class ColorPicker {
weak var delegate: ColorPickerDelegate?
func selectColor(_ color: String) {
   print("Selected color: \(color)")
   delegate?.didChooseColor(color)
   print("Color picker dismissed")
}
}
class HomeScreen: ColorPickerDelegate {
var backgroundColor = "White"
func didChooseColor(_ color: String) {
   backgroundColor = color
   print("Home screen background: \(backgroundColor)")
}
}
let home = HomeScreen()
let picker = ColorPicker()
picker.delegate = home

picker.selectColor("Red")
print("----------------")

picker.selectColor("Green")
print("----------------")
picker.selectColor("Blue")
```

output:

Output

```
Selected color: Red
Home screen background: Red
Color picker dismissed
----------------
Selected color: Green
Home screen background: Green
Color picker dismissed
----------------
Selected color: Blue
Home screen background: Blue
Color picker dismissed
```

Task 2: Practice Task, Lifecycle order across two screens:

```
import Foundation

class LoggingViewController {

let screenName: String

init(_ name: String) {

   screenName = name

}

func viewDidLoad() {

   print("\(screenName): viewDidLoad")

}

func viewWillAppear() {

   print("\(screenName): viewWillAppear")

}


func viewDidAppear() {

   print("\(screenName): viewDidAppear")

}


func viewWillDisappear() {

   print("\(screenName): viewWillDisappear")

}

func viewDidDisappear() {

   print("\(screenName): viewDidDisappear")
```

```
}
}
class ScreenA: LoggingViewController {
init() {
super.init("ScreenA")
}
}
class ScreenB: LoggingViewController {
init() {
super.init("ScreenB")
}
}
let screenA = ScreenA()
let screenB = ScreenB()
print("A. APP LAUNCH")
screenA.viewDidLoad()
screenA.viewWillAppear()
screenA.viewDidAppear()

print("\nB. PUSH SCREEN A TO SCREEN B")
screenA.viewWillDisappear()
screenB.viewDidLoad()
screenB.viewWillAppear()
screenA.viewDidDisappear()
screenB.viewDidAppear()

print("\nC. TAP BACK")
screenB.viewWillDisappear()
screenA.viewWillAppear()
screenB.viewDidDisappear()
screenA.viewDidAppear()
```

print("\nD. MODAL PRESENTATION")

screenA.viewWillDisappear()

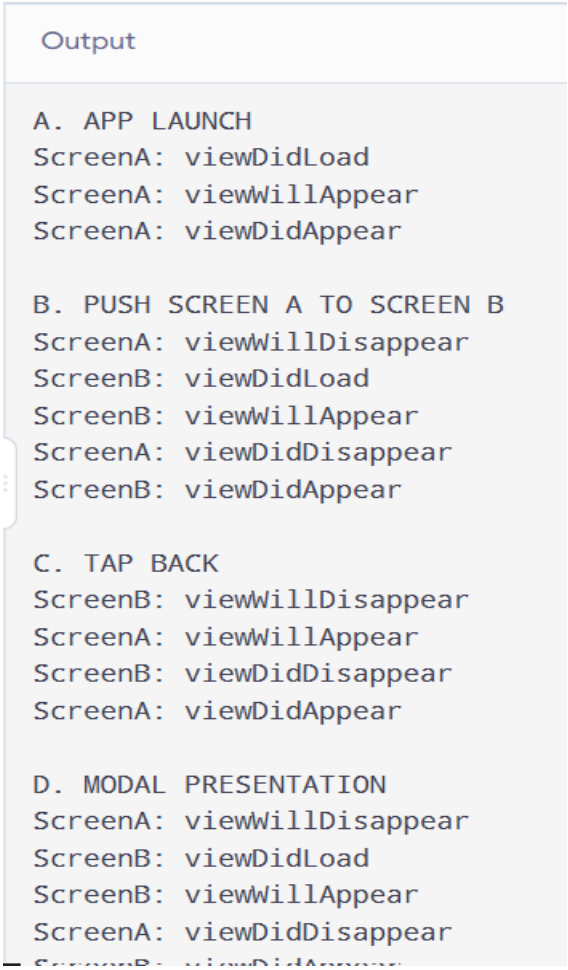
screenB.viewDidLoad()

screenB.viewWillAppear()

screenA.viewDidDisappear()

screenB.viewDidAppear()

output:



Task 3: Practice Task, UITextFieldDelegate:

import Foundation

class PhoneNumberValidator {
func validate(_ currentText: String, replacement: String) -> Bool {
let allowedCharacters = CharacterSet.decimalDigits
let isDigitsOnly = replacement.unicodeScalars.allSatisfy {

```
allowedCharacters.contains($0)
}
   if !isDigitsOnly {
      return false
   }
   return currentText.count + replacement.count <= 10
}
func shouldReturn() {
   print("Keyboard dismissed")
}
}
let validator = PhoneNumberValidator()
print("Enter phone number: 9876543210")
print("Digits only:", validator.validate("", replacement: "9876543210"))
print("Letters rejected:", !validator.validate("", replacement: "abc"))
print("More than 10 digits rejected:", !validator.validate("9876543210", replacement: "1"))
validator.shouldReturn()
```

output:

Output

```
Enter phone number: 9876543210
Digits only: true
Letters rejected: true
More than 10 digits rejected: true
Keyboard dismissed
```